2.4.4 Visualizing predictions by a Gaussian process

With this predictive Gaussian distribution in hand, we can now recreate the GP plots we have seen so far. Each of these plots consists of a mean function $\boldsymbol{\mu}$, which we could obtain from the multivariate Gaussian with:

```
predictive_mean = predictive_distribution.mean
```

Additionally, we want to show the 95% CI. Mathematically, this could be done by extracting the diagonal elements of the predictive covariance matrix Σ (remember these elements denote the individual variances σ^2), taking the square roots of these values to compute the standard deviations Σ, and finally computing the CI range of $\mu \pm 1.96\sigma$.

Fortunately for us, computing the 95% CI is a common operation when working with a GP, so GPyTorch offers a convenient helper method called `confidence_region()` that we may call directly from a multivariate Gaussian distribution object.

```
predictive_lower, predictive_upper =
<linearrow />    predictive_distribution.confidence_region()
```

This method returns a tuple of two Torch tensors, which store the lower and upper endpoints of the CI, respectively.

One last thing that we may want for our plot is samples drawn from our current GP model. We may directly do this by calling the method `sample()` from the Gaussian object `predictive_distribution`. If we don't pass in any input argument, the method will return a single sample. Here, we want to sample from our GP five times, which is done as follows:

```
torch.manual_seed(0)
samples = predictive_distribution.sample(torch.Size([5]))
```

We pass in a `torch.Size()` object to denote that we want five samples to be returned. Setting the random seed before sampling is a good practice to ensure reproducibility of our code. And with that, we are ready to make some plots!

The first thing to do is simply plotting the mean function:

```
plt.plot(xs, predictive_mean.detach(), label="mean")
```

As for the 95% CI, we typically use a shaded region like what we have seen so far, which can be done using Matplotlib's `fill_between()` function:

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>